《编译原理》书面作业

截止日期：2026 年 10 月 27 日

若发现问题，或有任何想法（改进题目、调整任务量等等），请及时联系助教

2026 年秋季第一次书面作业：自顶向下语法分析

除特别说明外，ϵ 表示空串，# 表示输入结束；空格仅分隔单词，不是输入的一部分。请写出必要的推导。

Q1. 回溯与预测

本题的 id、num 分别是标识符、数值常量的单词类别；标点均为独立单词。文法 $G_1[D]$ 描述一种声明或无参调用：

$$D \to id\ id\ \texttt{;} \mid id\ \texttt{(}\ \texttt{)}\ \texttt{;}$$

1. 对输入 id ( ) ; #，一个回溯分析器按书写顺序尝试产生式。指出第一次不匹配时的期望单词、实际单词，以及重试第二个候选式前输入位置应恢复到哪里；写出成功的最左推导。若不改写文法，预测分析所需的最少向前查看单词数是多少？说明理由。

2. 下面的 $G_2[S]$ 支持任意长度的限定名，例如 `a.b.c`。在展开 S 之前必须选定其产生式，且不允许回溯或改写文法。是否存在一个固定的有限 k，使查看接下来的至多 k 个单词总能做出正确选择？用一对具有任意长公共前缀的合法输入说明。

$$S \to A\ \texttt{=}\ num\ \texttt{;} \mid A\ \texttt{(}\ \texttt{)}\ \texttt{;}$$
$$A \to id\ \texttt{.}\ A \mid id$$

3. 将 G_2 改写为等价的 LL(1) 文法，并列出具有多个候选式的非终结符的各预测集合。解释改写后为什么只需一个向前查看单词。

Q2. 嵌套函数调用的 LL(1) 分析

考虑文法 $G[S]$。id 和 num 为单词类别，# 为输入结束标记，不属于文法的终结符集合。例如 `f(x, g(), 7);` 的单词序列为 id (id , id ( ) , num) ;。空实参表合法。

$$S \to id\ \texttt{(}\ A\ \texttt{)}\ \texttt{;}$$
$$A \to ER \mid \epsilon \qquad R \to \texttt{,}\ ER \mid \epsilon$$
$$E \to id\ C \mid num \quad C \to \texttt{(}\ A\ \texttt{)} \mid \epsilon$$

1. 求每个非终结符的 First、Follow 集合，并说明 E、C 的 Follow 集合如何得到。

2. 求每条产生式的预测集合，判断该文法是否为 LL(1)，并说明原因。

3. 构造完整的预测分析表，列为 $id, num, (,), ,, ;, \#$；用空白表示出错表项。

4. 对 `f(g());`，即 id (id ( )) ; $\#$，给出完整的表驱动分析过程。每行记录动作执行前的栈、剩余输入和下一步动作；栈顶在右侧，初始栈为 $\#S$。

Q3. 文法变换与 LL(1) 条件

1. 下面的文法以 E 为开始符号，描述标识符、无参调用及其后续下标访问，如 `a[0][1]`、`f()[2]`。方括号和圆括号都是终结符，num 表示一个数值单词。先消除直接左递归，再提取左公因子，给出等价的 LL(1) 文法；用各候选式的预测集合验证 LL(1) 条件。

$$E \to E \texttt{ [} num \texttt{]} \mid id \mid id \texttt{ ( )}$$

2. 对下面以 S 为开始符号的文法，严格按 S, A 的顺序执行课件中的一般左递归消除算法。写出代入后的产生式及最终文法。a, b, c, d 为终结符。

$$S \to Aa \mid b, \qquad A \to Sc \mid d$$

3. 第 2 小问得到的文法是否为 LL(1)? 至少给出一处预测集合冲突，并解释为什么 “已消除左递归且没有字面上的左公因子” 不足以保证 LL(1)。

Q4. 递归下降实现与语言扩展

沿用 Q2 的文法。词法分析器提供枚举值 `ID, NUM, LPAREN, RPAREN, COMMA, SEMI, END`，分别对应 $id, num, (,), ,, ;, \#$。全局变量 `lookahead` 已初始化为第一个单词。

`matchToken(t)` 在匹配时消费一个单词并更新 `lookahead`，否则调用 `syntaxError()`；`syntaxError()` 报错并立即终止，不返回。已提供正确的 `parseE()`：它按 Q2 的预测集合选择产生式，在 id 后调用 `parseC()`，对非法入口报错。

1. 写出 C 风格伪代码 `parseS()`、`parseA()`、`parseC()`。`parseS()` 是整个输入的入口，必须拒绝合法语句之后的多余单词；用循环处理逗号分隔的后续实参，不另写 `parseR()`。只有向前查看单词属于对应预测集合时才能选择 ϵ。简述程序为何拒绝 `f(x,);` 及 `f(); g();`。

2. 现在仅在本小问中允许非空实参表的末尾有一个可选的逗号，例如 `f(x,);` 和 `f(g(),);`。`f();` 仍合法，`f(,);` 和 `f(x,,);` 仍非法。直接添加 $R \to$ `,` 会造成什么预测集合冲突? 仅替换 R 的产生式并增加至多一个非终结符，给出满足要求的 LL(1) 文法片段及其预测集合。无需改写第 1 小问的程序。

Q5. 可复现的出错恢复

恢复使用 Q2 的原始文法（**不允许尾随逗号**）。保持原预测表中所有产生式不变，仅把原来为空且 $a \in \text{Follow}(X)$ 的表项 $M[X, a]$ 标为 `synch`。每步先查看栈顶，尚不弹栈；按以下规则恢复：

- 遇到 `synch`：报告缺少 X，弹出 X，不消费输入。

- 遇到其它空表项：报错一次；跳过输入直到遇到 $(\mathrm{First}(X) - \{\epsilon\}) \cup \mathrm{Follow}(X) \cup \{\#\}$ 中的单词。若该单词属于 First 则保留 X 并重试；否则若属于 Follow 则弹出 X；否则停止分析。不消费使跳过过程停止的那个单词，本次恢复不重复报错。
- 栈顶终结符 $t \neq \#$ 与输入不匹配时，报告缺少 t，弹出 t（视为插入），不消费输入。栈顶为 $\#$ 而输入未结束时，报错并丢弃剩余输入至 $\#$。两端均为 $\#$ 时结束。

1. 列出所有新增的 `synch` 表项。为什么不能将 Follow 列中的已有 ϵ 产生式改为 `synch`?
2. 输入为 `f(,x,);`，即 *id* `(` `,` *id* `,` `)` `;` $\#$。给出首次出错前的栈和剩余输入，并从该状态起逐步记录直到结束，格式与 Q2 相同；指出每次报错和恢复动作，并统计报错次数。
3. 输入为缺少分号的 `f()`，即 *id* `(` `)` $\#$。给出出错时的栈和恢复动作。对这两个非法输入，恢复后到达结束状态是否意味着原输入属于 $L(G)$？说明原因。

Fall 2026 Homework 1: Top-Down Parsing

Unless stated otherwise, ϵ denotes the empty string and # marks the end of input; spaces only separate tokens. Provide the necessary derivations.

Q1. Backtracking and prediction

Here *id* and *num* are identifier and numeric-literal token kinds; each punctuation mark is a separate token. Grammar $G_1[D]$ describes a declaration or a call with no arguments:

$$D \to id\ id\ \texttt{;} \mid id\ \texttt{(}\ \texttt{)}\ \texttt{;}$$

1. For input *id* `( ) ;` #, a backtracking parser tries productions in the written order. Give the expected and actual tokens at the first mismatch, the input position to restore before trying the second alternative, and the successful leftmost derivation. Without rewriting the grammar, what is the minimum lookahead for predictive parsing? Justify your answer.

2. Grammar $G_2[S]$ below supports arbitrarily long qualified names such as `a.b.c`. A production for S must be chosen before expanding it, without backtracking or rewriting the grammar. Can any fixed finite k make the next at most k tokens sufficient for every such choice? Exhibit two valid inputs with an arbitrarily long common prefix.

$$S \to A\ \texttt{=}\ num\ \texttt{;} \mid A\ \texttt{(}\ \texttt{)}\ \texttt{;}$$
$$A \to id\ \texttt{.}\ A \mid id$$

3. Rewrite G_2 as an equivalent LL(1) grammar. Give the prediction sets for every nonterminal with multiple alternatives, and explain why one-token lookahead now suffices.

Q2. LL(1) analysis of nested calls

Consider grammar $G[S]$. Symbols *id* and *num* are token kinds; # is the end marker and is not a grammar terminal. The token sequence for `f(x, g(), 7);` is *id* `(` *id* `,` *id* `(` `)` `,` *num* `)` `;`. Empty argument lists are valid.

$$S \to id\ \texttt{(}\ A\ \texttt{)}\ \texttt{;}$$
$$A \to ER \mid \epsilon \qquad R \to \texttt{,}\ ER \mid \epsilon$$
$$E \to id\ C \mid num \quad C \to \texttt{(}\ A\ \texttt{)} \mid \epsilon$$

1. Compute First and Follow for every nonterminal. Explain how the Follow sets of E and C arise.

2. Compute the prediction set of every production and justify whether the grammar is LL(1).

3. Construct the complete predictive parsing table with columns *id*, *num*, `(`, `)`, `,`, `;`, #; leave error entries blank.

4. Trace table-driven parsing of `f(g());`, namely *id* `(` *id* `(` `)` `)` `;` #. Each row must show the stack and remaining input before the next action, and that action. The stack top is on the right; initially the stack is $\#S$.

Q3. Grammar transformations and the LL(1) condition

1. The following grammar, with start symbol E, describes identifiers, calls with no arguments, and subsequent indexing, such as `a[0][1]` and `f()[2]`. Brackets and parentheses are terminals, and *num* is one numeric token. First eliminate direct left recursion, then left-factor to obtain an equivalent LL(1) grammar. Verify the LL(1) condition using the alternatives' prediction sets.

$$E \to E \texttt{ [} num \texttt{] } \mid id \mid id \texttt{ ( )}$$

2. For the grammar below, with start symbol S, apply the lecture's general left-recursion elimination algorithm in the order S, A. Show the productions after substitution and the final grammar. Symbols a, b, c, d are terminals.

$$S \to Aa \mid b, \qquad A \to Sc \mid d$$

3. Is the grammar obtained in part 2 LL(1)? Exhibit at least one prediction-set conflict and explain why removing left recursion and literal common left factors does not guarantee LL(1).

Q4. Recursive descent and a language extension

Use the grammar from Q2. The lexer provides enum values `ID, NUM, LPAREN, RPAREN, COMMA, SEMI, END`, corresponding to *id*, *num*, `(`, `)`, `,`, `;`, #. The global `lookahead` is initialized to the first token. `matchToken(t)` consumes a matching token and updates `lookahead`; otherwise it calls `syntaxError()`. `syntaxError()` reports an error and terminates immediately; it does not return. A correct `parseE()` is provided: it selects a production using Q2's prediction sets, calls `parseC()` after *id*, and reports invalid entry tokens.

1. Write C-style pseudocode for `parseS()`, `parseA()`, and `parseC()`. The entry point `parseS()` must reject extra tokens after one valid statement. Use a loop for the comma-separated remaining arguments instead of writing `parseR()`. Select ϵ only on a token in its prediction set. Briefly explain why the parser rejects `f(x,);` and `f(); g();`.

2. In this part only, allow one optional trailing comma in a nonempty argument list, as in `f(x,);` and `f(g(),);`. Keep `f();` valid and `f(,);` and `f(x,,);` invalid. What prediction-set conflict arises if we simply add $R \to$ `,`? Replace only the productions of R and introduce at most one new nonterminal to give an LL(1) grammar fragment with the required behavior and its prediction sets. You need not update the code from part 1.

Q5. Deterministic error recovery

Return to Q2's original grammar (**trailing commas are forbidden**). Preserve all production entries in its predictive table. Mark an originally empty entry $M[X, a]$ as `synch` only if $a \in \text{Follow}(X)$. Inspect the stack top before popping it, and use these recovery rules:

- On `synch`, report a missing X, pop X, and consume no input.

- On any other empty entry, report one error and skip input until reaching a token in $(\text{First}(X) - \{\epsilon\}) \cup \text{Follow}(X) \cup \{\#\}$. If it is in First, keep X and retry; otherwise, if it is in Follow, pop X; otherwise stop parsing. Do not consume the token that stops the skipping, and do not report the same recovery twice.
- If a terminal $t \neq \#$ on the stack mismatches the input, report missing t, pop t as an insertion, and consume no input. If the stack top is # before the input ends, report an error and discard the remaining input up to #. Finish when both are #.

1. List all added `synch` entries. Why must existing ϵ productions in Follow columns not be replaced by `synch`?
2. The input is `f(,x,);`, namely *id* `(` `,` *id* `,` `)` `;` #. Give the stack and remaining input immediately before the first error, then trace every step through completion in Q2's format. Identify every error and recovery action, and count the reported errors.
3. For `f()` with the semicolon missing, namely *id* `(` `)` #, give the stack at the error and the recovery action. For these two invalid inputs, does reaching the final state after recovery imply that the original input belongs to $L(G)$? Explain.